

实验5：基于自编码器的金融欺诈检测

1. 实验目标

- 熟悉并实操 PyTorch 中的自编码器（Autoencoder）建模流程
- 完成一个真实金融交易数据上的异常检测任务
 - 数据预处理 → 构建自编码器 → 训练 → 利用重构误差、DAGMM或其他基于自编码器的方法检测欺诈交易
- 正确理解不平衡数据集中的训练与评估方式
 - Precision、Recall、F1-score、AUC

2. 实验环境

- Python 版本：3.10+
- 必需库（建议版本）：
 - torch
 - numpy, pandas（数据处理）
 - scikit-learn（数据划分、指标计算）
 - matplotlib 或 seaborn（可视化）
- 建议使用 GPU 环境

3. 数据集

- **Credit Card Fraud Detection Dataset**
 - 来源：欧洲信用卡交易数据
 - 样本数：约 284,807 条
 - 欺诈交易比例 < 0.2%（极度不平衡）
- 下载地址：<https://www.kaggle.com/mlg-ulb/creditcardfraud>
- 划分方式：
 - 训练集：仅使用 **正常交易样本**（80%）
 - 验证+测试集：包含正常样本和欺诈样本（20%）

4. 实验任务

- 实现一个自编码器，实现信用欺诈数据集上的训练与预测。

- **要求**

- 仅使用正常样本训练 → 无监督异常检测
- 使用合适的重构误差阈值区分正常与异常
- 评价指标：Precision、Recall、F1-score、AUC

- **可选拓展：**

- 稀疏自编码器（加入 KL 散度或 L1 正则）
- 去噪自编码器（对输入加噪声）
- 不同瓶颈维度的对比
- 与逻辑回归 / 随机森林等传统监督方法比较
- 与其他异常检测方法比较

5. 实验步骤

1. 准备数据

- 下载 Kaggle 数据集并读取
- 归一化处理（如 StandardScaler）
- 训练集仅使用正常样本

2. 定义模型

- 实现 Encoder 与 Decoder
- 使用 `nn.MSELoss`

3. 训练模型

- 在正常样本上训练，保存验证集 loss 曲线
- 绘制训练/验证 loss

4. 测试与评估

- 在测试集上计算每个样本重构误差
- 设置阈值并输出检测结果
- 计算 Precision、Recall、F1-score、AUC
- 绘制 ROC 曲线和重构误差分布图

6. 实验报告要求

1. **摘要 (100–150 字)**：任务、方法、数据、主要结果
2. **方法**：自编码器的结构与训练方式，重构误差与阈值原理
3. **数据与预处理**：训练/验证/测试划分，归一化方法
4. **实验结果**：表格列出 Precision、Recall、F1、AUC
5. **可视化与分析**：必要的图片及分析
6. **结论与反思**：总结发现，分析不足与改进方向

7. 评分方式

- **实现正确性 (35)**：是否正确实现自编码器与异常检测流程
- **结果与可视化 (25)**：是否给出完整指标和曲线
- **分析深度 (20)**：是否完成至少一项消融/拓展实验并分析结果
- **报告质量 (10)**：表达清晰、结构完整、图表规范
- **加分项 (10)**：探索更好的方法，或与多种方法进行充分的比较